

SAL: Multi-modal Verification of Replicated Data Types

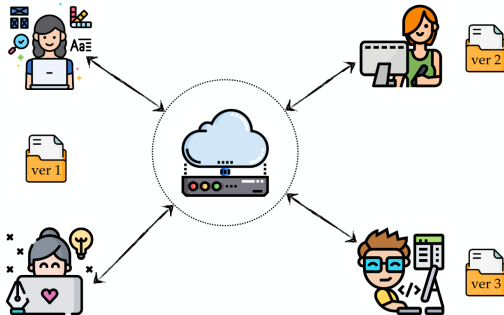
Pranav R Vimala S KC Sivaramakrishnan

Indian Institute of Technology, Madras

April 23, 2026

Local-First Collaboration

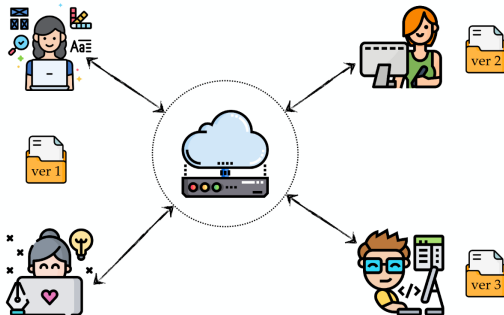
- Users work offline, synchronize when connectivity returns



Challenge: How do we ensure correctness when states diverge?

Local-First Collaboration

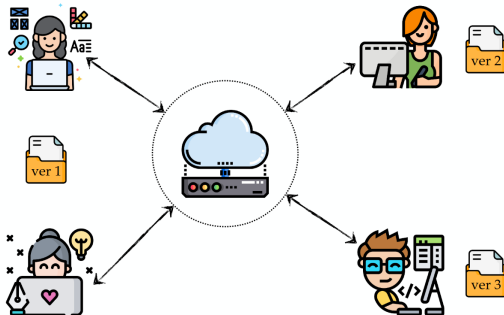
- Users work offline, synchronize when connectivity returns
- Replicas evolve independently, must merge accurately



Challenge: How do we ensure correctness when states diverge?

Local-First Collaboration

- Users work offline, synchronize when connectivity returns
- Replicas evolve independently, must merge accurately
- Example: Shared documents, collaborative editors



Challenge: How do we ensure correctness when states diverge?

CRDTs and MRDTs: Building Blocks

CRDT Conflict-Free Replicated Data Type

- Deterministic two-way merge

MRDT Mergeable Replicated Data Type

MERGE STRATEGIES

The merge mechanism (`git merge` and `git pull` commands) allows the backend **merge strategies** to be chosen with `-s` option. Some strategies can also take their own options, which can be passed by giving `-X<option>` arguments to `git merge` and/or `git pull`.

ort

This is the default merge strategy when pulling or merging one branch. This strategy can only resolve two heads using a **3-way merge algorithm**. When there is more than one common ancestor that can be used for 3-way

Three-way merge model in git (git-ort)

CRDTs and MRDTs: Building Blocks

CRDT Conflict-Free Replicated Data Type

- Deterministic two-way merge
- Requires tombstones

MRDT Mergeable Replicated Data Type

MERGE STRATEGIES

The merge mechanism (`git merge` and `git pull` commands) allows the backend **merge strategies** to be chosen with `-s` option. Some strategies can also take their own options, which can be passed by giving `-X<option>` arguments to `git merge` and/or `git pull`.

ort

This is the default merge strategy when pulling or merging one branch. This strategy can only resolve two heads using a **3-way merge algorithm**. When there is more than one common ancestor that can be used for 3-way

Three-way merge model in git (git-ort)

CRDTs and MRDTs: Building Blocks

CRDT Conflict-Free Replicated Data Type

- Deterministic two-way merge
- Requires tombstones

MRDT Mergeable Replicated Data Type

- Three-way merge (like Git)

MERGE STRATEGIES

The merge mechanism (`git merge` and `git pull` commands) allows the backend `merge strategies` to be chosen with `-s` option. Some strategies can also take their own options, which can be passed by giving `-X<option>` arguments to `git merge` and/or `git pull`.

ort

This is the default merge strategy when pulling or merging one branch. This strategy can only resolve two heads using a `3-way merge algorithm`. When there is more than one common ancestor that can be used for 3-way

Three-way merge model in git (git-ort)

CRDTs and MRDTs: Building Blocks

CRDT Conflict-Free Replicated Data Type

- Deterministic two-way merge
- Requires tombstones

MRDT Mergeable Replicated Data Type

- Three-way merge (like Git)
- More compact representations (e.g., $O(1)$ counter state vs $\Omega(n)$ in CRDTs due to tombstones)

MERGE STRATEGIES

The merge mechanism (`git merge` and `git pull` commands) allows the backend `merge strategies` to be chosen with `-s` option. Some strategies can also take their own options, which can be passed by giving `-X<option>` arguments to `git merge` and/or `git pull`.

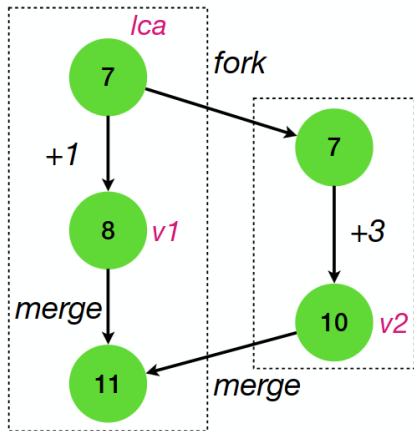
ort

This is the default merge strategy when pulling or merging one branch. This strategy can only resolve two heads using a `3-way merge algorithm`. When there is more than one common ancestor that can be used for 3-way

Three-way merge model in git (git-ort)

Example: Increment-Only Counter MRDT

Component	Definition
State	$\Sigma = \mathbb{N}, \sigma_0 = 0$
Initial	$\sigma_0 = 0$
Update	$\text{do}(\sigma, \dots, \text{inc}) = \sigma + 1$
Merge	$\text{merge}(\sigma_{lca}, \sigma_1, \sigma_2)$ $= \sigma_{lca} + (\sigma_1 - \sigma_{lca})$ $+ (\sigma_2 - \sigma_{lca})$
Conflicts	$\text{rc} = \emptyset$ (none)



Three-way merge after concurrent increments

Why Verification Matters

Designing correct replicated data types is challenging.

Errata

A few years after publication, we learned about some serious flaws in this paper:

- › The definition of non-interleaving in Section 2.1 cannot be guaranteed by any algorithm.
- › The algorithm presented in Section 3.1 does not guarantee convergence.

Please see the [Fugue paper](#) for more details.

RGa anomaly: The definition of non-interleaving in that paper cannot be satisfied by any algorithm

5.2 Buggy MRDT Implementation in [23]

We now present some details of one of the buggy MRDTs, Enable-wins flag, that we discovered using our framework in the work by Soundarapandian et al. [23]. The state of the enable-wins flag MRDT consists of a pair: a counter and a flag. The counter tracks the number of the enable events, while the flag is set to true on an enable event. The desired specification for this flag is that it should be true when there is at least one enable event not visible to any disable event.

Enable-wins flag bug: Incorrect merge logic allows disabled state to incorrectly become enabled

Replication-Aware (RA) Linearizability

Definition

RA linearizability relates replica executions to a sequential explanation of updates, ensuring that:

- The state at any replica during any execution must be achievable by applying a sequence (or linearization) of the update operations visible to the replica.

Neem reduced RA-linearizability to 24 verifiable conditions (VCs)

Replication-Aware (RA) Linearizability

Definition

RA linearizability relates replica executions to a sequential explanation of updates, ensuring that:

- The state at any replica during any execution must be achievable by applying a sequence (or linearization) of the update operations visible to the replica.
- This linearization should obey the user-specified conflict resolution policy for concurrent operations, and the local replica order for non-concurrent operations.

Neem reduced RA-linearizability to 24 verifiable conditions (VCs)

Replication-Aware (RA) Linearizability

Definition

RA linearizability relates replica executions to a sequential explanation of updates, ensuring that:

- The state at any replica during any execution must be achievable by applying a sequence (or linearization) of the update operations visible to the replica.
- This linearization should obey the user-specified conflict resolution policy for concurrent operations, and the local replica order for non-concurrent operations.
- Our definition of RA-linearizability allows viewing the state of an MRDT replica as a sequence of update operations applied on the initial state, thus abstracting over the merge function and how it handles concurrent operations.

Neem reduced RA-linearizability to 24 verifiable conditions (VCs)

Prior Work: Neem in F^*

Strengths

- SMT-aided automation for RA linearizability

The Problem: Trust in SMT Solvers

Prior Work: Neem in F[★]

Strengths

- SMT-aided automation for RA linearizability
- Formal reduction to 24 verification conditions

The Problem: Trust in SMT Solvers

Prior Work: Neem in F^*

Strengths

- SMT-aided automation for RA linearizability
- Formal reduction to 24 verification conditions
- Practical verification of CRDTs and MRDTs

The Problem: Trust in SMT Solvers

Prior Work: Neem in F^*

Strengths

- SMT-aided automation for RA linearizability
- Formal reduction to 24 verification conditions
- Practical verification of CRDTs and MRDTs

The Problem: Trust in SMT Solvers

- SMT-centric workflows are opaque when automation fails

Prior Work: Neem in F^*

Strengths

- SMT-aided automation for RA linearizability
- Formal reduction to 24 verification conditions
- Practical verification of CRDTs and MRDTs

The Problem: Trust in SMT Solvers

- SMT-centric workflows are opaque when automation fails
- Failed VC provides little actionable feedback

Prior Work: Neem in F[★]

Strengths

- SMT-aided automation for RA linearizability
- Formal reduction to 24 verification conditions
- Practical verification of CRDTs and MRDTs

The Problem: Trust in SMT Solvers

- SMT-centric workflows are opaque when automation fails
- Failed VC provides little actionable feedback
- Relies on external solver → larger trusted computing base (TCB)

Prior Work: Neem in $F^\star$

Strengths

- SMT-aided automation for RA linearizability
- Formal reduction to 24 verification conditions
- Practical verification of CRDTs and MRDTs

The Problem: Trust in SMT Solvers

- SMT-centric workflows are opaque when automation fails
- Failed VC provides little actionable feedback
- Relies on external solver $\rightarrow$ larger trusted computing base (TCB)
- Debugging requires manual proof-debugging workflow

Prior Work: Neem in F^*

Strengths

- SMT-aided automation for RA linearizability
- Formal reduction to 24 verification conditions
- Practical verification of CRDTs and MRDTs

The Problem: Trust in SMT Solvers

- SMT-centric workflows are opaque when automation fails
- Failed VC provides little actionable feedback
- Relies on external solver $\rightarrow$ larger trusted computing base (TCB)
- Debugging requires manual proof-debugging workflow
- SMT proofs are brittle to small changes

A Multi-Modal Workflow for RA Linearizability in Lean

1 Kernel-checkable automation with proof reconstruction

16. The `grind` tactic

The `grind` tactic uses techniques inspired by modern SMT solvers to automatically construct proofs. It produces proofs by incrementally collecting sets of facts, deriving new facts from the existing ones using a set of cooperating techniques. Behind the scenes, all proofs are by contradiction, so there is no operational distinction between the expected conclusion and the premises; `grind` always attempts to derive a contradiction.

Like other tactics, `grind` produces ordinary Lean proof terms for every fact it adds. Lean's standard library is already annotated with `@[grind]` attributes, so common lemmas are discovered automatically.

A Multi-Modal Workflow for RA Linearizability in Lean

- 1 **Kernel-checkable automation** with proof reconstruction
- 2 **SMT-aided automation** when needed (fallback)

16. The `grind` tactic

The `grind` tactic uses techniques inspired by modern SMT solvers to automatically construct proofs. It produces proofs by incrementally collecting sets of facts, deriving new facts from the existing ones using a set of cooperating techniques. Behind the scenes, all proofs are by contradiction, so there is no operational distinction between the expected conclusion and the premises; `grind` always attempts to derive a contradiction.

Like other tactics, `grind` produces ordinary Lean proof terms for every fact it adds. Lean's standard library is already annotated with `@[grind]` attributes, so common lemmas are discovered automatically.

A Multi-Modal Workflow for RA Linearizability in Lean

- 1 **Kernel-checkable automation** with proof reconstruction
- 2 **SMT-aided automation** when needed (fallback)
- 3 **AI-assisted interactive theorem proving** for remaining obligations

16. The `grind` tactic

The `grind` tactic uses techniques inspired by modern SMT solvers to automatically construct proofs. It produces proofs by incrementally collecting sets of facts, deriving new facts from the existing ones using a set of cooperating techniques. Behind the scenes, all proofs are by contradiction, so there is no operational distinction between the expected conclusion and the premises; `grind` always attempts to derive a contradiction.

Like other tactics, `grind` produces ordinary Lean proof terms for every fact it adds. Lean's standard library is already annotated with `@[grind]` attributes, so common lemmas are discovered automatically.

A Multi-Modal Workflow for RA Linearizability in Lean

- 1 **Kernel-checkable automation** with proof reconstruction
- 2 **SMT-aided automation** when needed (fallback)
- 3 **AI-assisted interactive theorem proving** for remaining obligations
- 4 **Counterexample generation** and visualization

16. The `grind` tactic

The `grind` tactic uses techniques inspired by modern SMT solvers to automatically construct proofs. It produces proofs by incrementally collecting sets of facts, deriving new facts from the existing ones using a set of cooperating techniques. Behind the scenes, all proofs are by contradiction, so there is no operational distinction between the expected conclusion and the premises; `grind` always attempts to derive a contradiction.

Like other tactics, `grind` produces ordinary Lean proof terms for every fact it adds. Lean's standard library is already annotated with `@[grind]` attributes, so common lemmas are discovered automatically.

A Multi-Modal Workflow for RA Linearizability in Lean

- 1 **Kernel-checkable automation** with proof reconstruction
- 2 **SMT-aided automation** when needed (fallback)
- 3 **AI-assisted interactive theorem proving** for remaining obligations
- 4 **Counterexample generation** and visualization
- 5 **The grind tool in Lean offers kernel-checkable proofs**

16. The `grind` tactic

The `grind` tactic uses techniques inspired by modern SMT solvers to automatically construct proofs. It produces proofs by incrementally collecting sets of facts, deriving new facts from the existing ones using a set of cooperating techniques. Behind the scenes, all proofs are by contradiction, so there is no operational distinction between the expected conclusion and the premises; `grind` always attempts to derive a contradiction.

Like other tactics, `grind` produces ordinary Lean proof terms for every fact it adds. Lean's standard library is already annotated with `@[grind]` attributes, so common lemmas are discovered automatically.

Key Design: Data Structures for Automation

Challenge: Lean's standard library optimizes for manual proofs, not automation

Custom Set and Map Interfaces

```
-- Set: Boolean-valued membership
@[simp]
abbrev set (a:Type) [DecidableEq a] := a -> Bool

@[simp, grind]
def union {a:Type} [DecidableEq a] (s1 s2 : set a) : set a :=
  (fun x => s1 x || s2 x)

@[simp, grind]
def intersection {a:Type} [DecidableEq a] (s1 s2: set a) : set a :=
  (fun x => s1 x && s2 x)

-- Map: Explicit domain
structure map (key:Type) [DecidableEq key] (value:Type) where
  (mappings: key -> value) (domain: set key)
```

Automation-friendly: All operations are directly computable!

The Enable-Wins Flag Bug

Specification

A shared boolean flag with concurrent enable/disable operations. When enable and disable are concurrent, **enable wins**.

Naive Implementation Bug:

- Track concurrent enables with a counter

The Enable-Wins Flag Bug

Specification

A shared boolean flag with concurrent enable/disable operations. When enable and disable are concurrent, **enable wins**.

Naive Implementation Bug:

- Track concurrent enables with a counter
- Use counter to decide flag state after merge

The Enable-Wins Flag Bug

Specification

A shared boolean flag with concurrent enable/disable operations. When enable and disable are concurrent, **enable wins**.

Naive Implementation Bug:

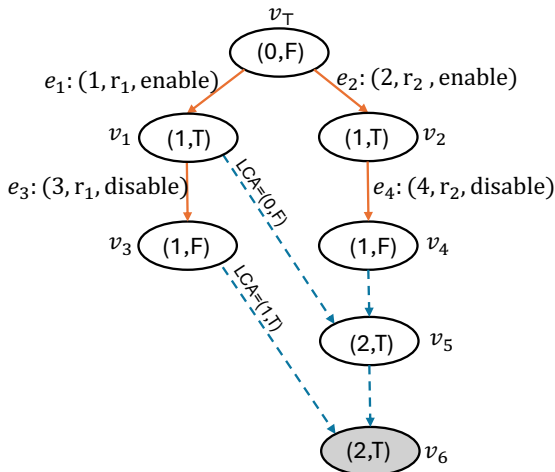
- Track concurrent enables with a counter
- Use counter to decide flag state after merge
- **Problem:** Enables followed by disables on each replica still show as enabled!

RA-Linearizability Verification Condition (VC)

```
theorem inter_right_lop (l: concrete_st) (a: concrete_st)
  (b: concrete_st) (o1: op_t) (ob: op_t) (o1: op_t) (o: op_t) :
  rc ob o1 = rc_res.Fst_then_snd /\ get_rid ob != get_rid o1 /\
  (~ (rc o ob = rc_res.Either) \/ (rc o o1 = rc_res.Fst_then_snd)) /\
  distinct_ops o1 ob /\ distinct_ops o1 o1 /\
  distinct_ops o1 o /\ distinct_ops ob o1 /\
  distinct_ops ob o /\ distinct_ops o1 o /\
  get_rid o != get_rid o1 /\
  eq (merge (do_ l o1) (do_ (do_ a o1) o1)
      (do_ (do_ b ob) o1))
      (do_ (merge (do_ l o1) (do_ a o1)
                  (do_ (do_ b ob) o1)) o1)
  ->
  eq (merge (do_ l o1) (do_ (do_ a o1) o1)
      (do_ (do_ (do_ b o) ob) o1))
      (do_ (merge (do_ l o1) (do_ a o1)
                  (do_ (do_ (do_ b o) ob) o1)) o1)
```

Conditions ensure interleaving property: operations commute correctly across merge boundaries

Concrete Counterexample: Enable-Wins Flag Execution



Automatic counterexample shows: both replicas disable the flag (v_3, v_4), yet merge produces $(2, \text{true})$ —flag is incorrectly enabled!

Automated Counterexample Discovery

Approach: Use Plausible (property-based testing for Lean)

Process

- 1 Express VCs as **decidable propositions**

Example failing input:

```
(disable, (enable, (enable, (disable, (0, false)))))
```

Automated Counterexample Discovery

Approach: Use Plausible (property-based testing for Lean)

Process

- 1 Express VCs as **decidable propositions**
- 2 Plausible generates random test inputs

Example failing input:

```
(disable, (enable, (enable, (disable, (0, false)))))
```

Automated Counterexample Discovery

Approach: Use Plausible (property-based testing for Lean)

Process

- 1 Express VCs as **decidable propositions**
- 2 Plausible generates random test inputs
- 3 Check whether VC holds for each input

Example failing input:

```
(disable, (enable, (enable, (disable, (0, false)))))
```

Automated Counterexample Discovery

Approach: Use Plausible (property-based testing for Lean)

Process

- 1 Express VCs as **decidable propositions**
- 2 Plausible generates random test inputs
- 3 Check whether VC holds for each input
- 4 When violation found: report the failing input

Example failing input:

```
(disable, (enable, (enable, (disable, (0, false)))))
```

Automated Counterexample Discovery

Approach: Use Plausible (property-based testing for Lean)

Process

- 1 Express VCs as **decidable propositions**
- 2 Plausible generates random test inputs
- 3 Check whether VC holds for each input
- 4 When violation found: report the failing input
- 5 Uses machinery from the Loom work from VERSE lab

Example failing input:

```
(disable, (enable, (enable, (disable, (0, false)))))
```

Plausible: Making VCs Testable

- 1 **Proving Precondition Decidable** by upper bounding the values of the operations and counters, and subsequently using `decidable_by_nat_upperbound`

```
extract_program_for inter_right_lop
prove_precondition_decidable_for inter_right_lop by

  dsimp
  cases (ob.2.2)
  {
    cases (ol.2.2)
    {
      simp
      exact (decidable_by_nat_upperbound [])
    }
  }
```

Plausible: Making VCs Testable

- 1 **Proving Precondition Decidable** by upper bounding the values of the operations and counters, and subsequently using `decidable_by_nat_upperbound`

```
extract_program_for inter_right_lop
prove_precondition_decidable_for inter_right_lop by

  dsimp
  cases (ob.2.2)
  {
    cases (ol.2.2)
    {
      simp
      exact (decidable_by_nat_upperbound [])
    }
  }
```

- 2 **Proving Postcondition Decidable** again by using `decidable_by_nat_upperbound`

```
prove_postcondition_decidable_for inter_right_lop by
  dsimp
  exact (decidable_by_nat_upperbound [])
  derive_tester_for inter_right_lop
```

Plausible: Running the Elaborator

③ Running the Elaborator in Monadic Framework

Running an elaborator in a monadic framework to generate and display the counterexample

```
postcondition violated for input (false, (true, (true, (false, ((3, true), ((3, true), (3, false))))))
Lean 4
```

The `run_elab doSeq` command executes code in `TermElabM Unit`. This is the same as `#eval show TermElabM Unit from discard do doSeq`.

[View Problem \(Alt+F8\)](#) [Fix](#)

```
run_elab do
  let g: Plausible.Gen ( _ × Bool ) := do
    let x1 ← Plausible.SampleableExt.interpSample (Bool)
    let x2 ← Plausible.SampleableExt.interpSample (Bool)
    let x3 ← Plausible.SampleableExt.interpSample (Bool)
    let x4 ← Plausible.SampleableExt.interpSample (Bool)
    let y1 ← Plausible.SampleableExt.interpSample (concrete_st)
    let y1' ← Plausible.SampleableExt.interpSample (concrete_st)
    let y1'' ← Plausible.SampleableExt.interpSample (concrete_st)
    let y2 ← Plausible.SampleableExt.interpSample (N)
    let res := inter_right_lopTester y1 y1' y1'' (y2,y2+1,myfun (x1)) (y2+2,y2+3, m
    pure ((x1,x2,x3,x4,y1,y1', y1''),res)
  for _ in [1:500] do
    let res ← Plausible.Gen.run g 3
    unless res.2 do
      IO.println s!"postcondition violated for input {res.1}"
      break
```


The Challenge

Raw counterexample is hard to understand—why does it fail?

Solution: ProofWidgets-based execution trace visualizer

- Instrument do and merge operations

Transforms opaque proof failures into tangible test cases

The Challenge

Raw counterexample is hard to understand—why does it fail?

Solution: ProofWidgets-based execution trace visualizer

- Instrument do and merge operations
- Record intermediate states and operations

Transforms opaque proof failures into tangible test cases

The Challenge

Raw counterexample is hard to understand—why does it fail?

Solution: ProofWidgets-based execution trace visualizer

- Instrument do and merge operations
- Record intermediate states and operations
- Produce step-by-step execution trace

Transforms opaque proof failures into tangible test cases

The Challenge

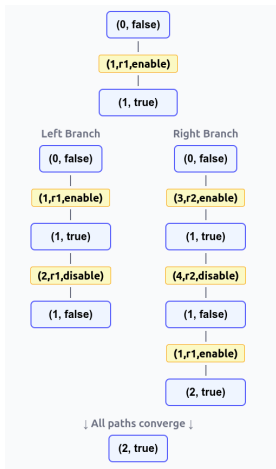
Raw counterexample is hard to understand—why does it fail?

Solution: ProofWidgets-based execution trace visualizer

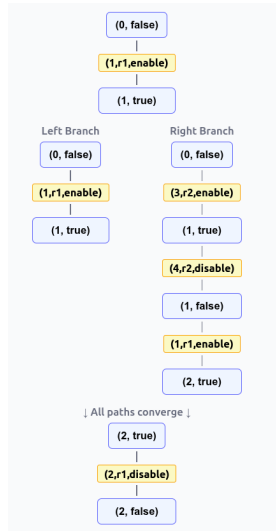
- Instrument do and merge operations
- Record intermediate states and operations
- Produce step-by-step execution trace
- Interactive visualization in Lean

Transforms opaque proof failures into tangible test cases

Enable-Wins Flag: Verification Conditions



LHS (merge result)



RHS (expected after linearization)

Stepping Back

SAL is more than tactics—it's a collaborative verification workflow:

- **Kernel automation:** Handles routine reasoning

Stepping Back

SAL is more than tactics—it's a collaborative verification workflow:

- **Kernel automation:** Handles routine reasoning
- **SMT when needed:** For harder goals

Stepping Back

SAL is more than tactics—it's a collaborative verification workflow:

- **Kernel automation:** Handles routine reasoning
- **SMT when needed:** For harder goals
- **AI assistance:** Proofs from Aristotle and Claude

Multi-Modal Verification: Agentic Framework

Stepping Back

SAL is more than tactics—it's a collaborative verification workflow:

- **Kernel automation:** Handles routine reasoning
- **SMT when needed:** For harder goals
- **AI assistance:** Proofs from Aristotle and Claude
- **Developer:** Makes informed decisions based on counterexample and proof strategies given by LLMs

AI-Assisted Verification: Agentic-Driven Commits

LWW_Map_CRDT: close remaining 4 VCs via Aristotle-generated intermediate lemmas

33e5fc9  

 kayceesrk committed 13 hours ago ·  1 / 1

add Peritext_CRDT (first pass, 15/24 VCs closed, 9 TODO) ...

7b435af  

kayceesrk committed 16 hours ago · ✓ 1 / 1

Commits on Apr 21, 2026

add RGA_CRDT (state-based sequence CRDT; Peritext dependency) ...

c3fcf49  

 kayceesrk committed yesterday · ✓ 1 / 1

add Bounded_Counter_CRDT (Sypytkowski 2019 / Balegas et al. 2015) ...

5116447  

 kayceesrk committed yesterday · ✓ 1 / 1

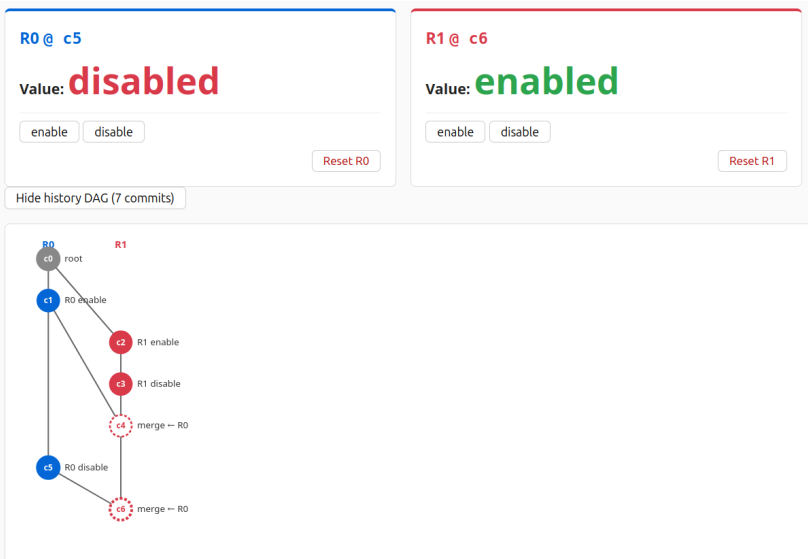
Merge pull request #1 from fplaunchpad/wip/bump-to-4.28

Verified

5e4a1b0  

 kayceesrk authored yesterday ·  1 / 1

AI-Assisted Verification: NodeJs Interactive Simulations



AI-Assisted Verification: Verification Status

The approach builds on the $F^\star$ -based **Neem** framework of Soundarapandian, Nagar, Rastogi, and Sivaramakrishnan (OOPSLA 2025), which reduces RA-linearizability for a data type to a fixed set of VCs over `do`, `merge`, and `rc`. Sal is the Lean port of that reduction plus a multi-modal tactic, counterexample pipeline, and custom decidable set/map interfaces that make `grind` effective on RDT goals.

What's verified

The suite currently contains **29 RDTs** — 18 CRDTs and 11 MRDTs — with all 24 RA-linearizability VCs closed on every one (696 kernel-verified VCs in total). Everything is checked on Lean `v4.28.0` against the `chore-bump-lean-4.28` branch of a [Blaster fork](#).

Results Table

Table: Number of VCs discharged by the SAL tactic (total VCs = 24 per RDT). DG refers to dsimp + grind, and LB refers to lean – blaster.

RDT	DG	LB	ITP (Aristotle)	Time (s)
Increment-only counter MRDT	24	0	0	0.7
PN-counter MRDT	24	0	0	2.7
OR-set MRDT	3	21	0	17.3
Enable-wins flag MRDT	9	14	0	34.2
Efficient OR-set MRDT	2	22	0	14.4
Grows-only set MRDT	24	0	0	0.3
Grows-only map MRDT	22	0	2	25.8
Replicated growable array MRDT	15	9	0	3.9
Multi-valued register MRDT	24	0	0	0.3
Increment-only counter CRDT	24	0	0	1.5
PN-counter CRDT	16	2	6	99.5
Multi-valued register CRDT	24	0	0	0.7
OR-set CRDT	4	19	1	34.7

Thank you!

Questions?